

COMPUTER ARCHITECTURE ASSIGNMENT

Branch Prediction Exploration & Implementation using ChampSim

Name: Tanishka Trivedi

Roll no.: B24EE1080

OBJECTIVE:

The objective of this assignment is to familiarize students with ChampSim, understand existing branch prediction mechanisms, evaluate their performance, and implement progressively more sophisticated branch predictors. To work with ChampSim to:

- Explore existing branch predictors.
- Understand and compare branch-prediction performance.
- Implement a 2-bit saturating counter predictor.
- Design and evaluate a table-based branch predictor with different table sizes.

STEP-01: Explore Existing Branch Predictors

1.1 Bimodal Predictor:

The bimodal branch predictor is one of the simplest forms of branch prediction used in modern processors to guess whether a branch will be taken or not taken. Its simplicity lies in maintaining a single bit or two bits of state for each branch instruction in the program, which is indexed by part of the instruction's address. The predictor maintains a table of 16K entries, with each entry storing a saturating counter whose value ranges from 0 to 3. The branch instruction address (IP) is mapped to an entry using $IP \% 16381$. A counter value of 0 or 1 predicts Not Taken, while 2 or 3 predicts Taken. After the actual branch outcome is known, the counter is incremented for a Taken branch and decremented for a Not-Taken branch, with saturation at the maximum value 3 and minimum value 0.

1.2 GShare Predictor:

Modern processors thrive on speed and efficiency, but predicting the flow of instructions in a program is a significant challenge. Enter branch prediction, a cornerstone of high-performance computing. Among various strategies, the Global History Register

(GHR) branch predictor stands out for its ability to track patterns across multiple branches, boosting prediction accuracy. At its core, branch prediction aims to guess the outcome of a branch (e.g., an if statement) before it executes. Predicting the right path ensures the processor doesn't stall waiting for the outcome, keeping instruction pipelines humming.

The key difference from Bimodal is that GShare doesn't use only the branch PC. It combines the branch address with global branch history when selecting the predictor-table entry. The GShare predictor maintains a 14-bit global branch history and a 16K-entry table of 2-bit saturating counters. The table index is generated by combining the branch instruction address with the global branch history using XOR-based hashing. A counter value of 2 or 3 predicts Taken, while 0 or 1 predicts Not Taken. After the actual branch outcome is known, the corresponding counter is incremented or decremented with saturation at 3 and 0. The global history is then shifted and updated with the latest branch outcome.

1.3 Perceptron Predictor:

A perceptron branch predictor is a type of dynamic branch predictor in computer architecture that uses a single-layer artificial neural network (the perceptron) instead of traditional saturating counters or pattern history tables to guess whether a conditional branch will be taken. It has one limitation though- its latency. Computing the weighted sum across a large history length takes more time than a simple table lookup, introducing higher inference latency that often requires pipelining or overriding hybrid designs to hide the delay.

1.4 Identify (The table!)

<i>Predictor</i>	<i>Predictor State</i>	<i>Number of entries</i>	<i>Indexing Mechanism</i>	<i>Update Mechanism</i>	<i>Prediction Mechanism</i>
Bimodal	2-bit saturating counters	16,384 (16K)	IP % 16381	Increment for Taken, decrement for Not Taken; saturates at	Counter $\geq 2 \rightarrow$ Taken; otherwise Not Taken

				0/3	
Gshare	14-bit global history + 2-bit saturating counters	16,384 (16K)	XOR-based hash of IP and global history, then modulo 16,384	Increment/decrement counter based on outcome; update global history	Counter $\geq 2 \rightarrow$ Taken; otherwise Not Taken
Perceptron	24-bit global history + 25 weights per perceptron (1 bias + 24 history weights), 8-bit signed weights	163 perceptrons (These aren't 163 simple counters. Each entry is a perceptron containing 25 weights, and each weight is 8-bit signed.)	IP % 163	Train bias/history weights when prediction is incorrect or insufficiently confident; weights saturate at -128/+127	Weighted sum $\geq 0 \rightarrow$ Taken; $< 0 \rightarrow$ Not Taken

1.5 How do predictors differ?

The Bimodal predictor is the simplest of the three and uses a 2-bit saturating counter indexed primarily by the branch instruction address, without maintaining global branch history. GShare extends the counter-based approach by incorporating a 14-bit global branch history into the table-indexing process through an XOR-based hash, allowing it to capture relationships between recent branch outcomes and the current branch. The Perceptron predictor uses a different approach: instead of a single 2-bit counter, it maintains multiple weighted perceptrons and uses a 24-bit global history to calculate a weighted sum for prediction. Thus, the three predictors progressively differ in the amount and type of historical information used and in the complexity of their prediction mechanism.

Step-02: Compare Existing Predictors

To run the selected branch predictors using the same traces and simulation configuration. Then, collect and compare- IPC and simulation cycles. Using the same configuration, warmup period, simulation period, and traces for all predictors.

```

Problems  Output  Debug Console  Terminal  Ports
Compiling prefetcher/l2c_prefetcher.cc...
Generating dependencies for prefetcher/l1c_prefetcher.cc...
Compiling prefetcher/l1c_prefetcher.cc...
Linking bin/champsim...

ChampSim is successfully built
Branch Predictor: bimodal
L1I Prefetcher: no
L1D Prefetcher: no
L2C Prefetcher: no
LLC Prefetcher: no
LLC Replacement: lru
Cores: 1
Binary: bin/bimodal-no-no-no-no-lru-1core

tanishkatrivedi@HP:/mnt/c/Users/Tanishka/Downloads/ChampSim-master/ChampSim-master$

```

(showing that ChampSim successfully built bimodal- similarly done for Gshare and Perceptron)

2. 1 EXPERIMENTAL TABLE: (also in .csv in repo)

Trace	Predictor	IPC	Total Branches	Correct Predictions*	Mispredictions*	Accuracy	Mispred. Rate*	L1I Miss Rate	L1D Miss Rate	LLC Miss Rate
602.gcc_s	Bimodal	0.2820	2,563,586	~2,543,762	~19,824	92.2669%	~0.7729%	0.00%	8.0303%	99.7548%
602.gcc_s	GShare	0.3184	2,563,591	~2,554,529	~9,062	96.4651%	~0.3534%	0.00%	8.2703%	99.7536%
602.gcc_s	Perceptron	0.3198	2,563,591	~2,554,910	~8,681	96.6137%	~0.3387%	0.00%	8.2564%	99.7536%
605.mcf_s	Bimodal	0.3298	2,483,269	~2,458,563	~24,706	90.0509%	~0.9950%	0.00%	14.2181%	42.9752%
605.mcf_s	GShare	0.3288	2,483,280	~2,462,962	~20,318	91.8183%	~0.8186%	0.00%	14.3251%	42.9949%

605.mcf_s	Perceptron	0.3483	2,483,280	~2,465,231	~18,049	92.7317%	~0.7270%	0.00%	14.4001%	42.9949%
607.cactuBSSN_s	Bimodal	1.2246	91,036	~90,976	~60	99.3398%	~0.0660%	18.6137%	18.6907%	47.9713%
607.cactuBSSN_s	GShare	1.2252	91,036	~90,997	~39	99.5694%	~0.0431%	18.6120%	18.6922%	47.8652%
607.cactuBSSN_s	Perceptron	1.2251	91,036	~91,001	~35	99.6166%	~0.0383%	18.6133%	18.6920%	47.8660%

2.2 ANALYSIS:

Q1. How does branch prediction accuracy vary between predictors?

Ans: Branch prediction accuracy varies considerably across workloads. For 602.gcc_s, accuracy improves from 92.27% with Bimodal to 96.47% with GShare and 96.61% with Perceptron. A similar trend is observed for 605.mcf_s, where accuracy increases from 90.05% to 91.82% and 92.73%, respectively. However, for 607.cactuBSSN_s, all three predictors achieve approximately 99.3–99.6% accuracy, showing that predictor complexity provides only a small benefit for this workload. Thus, incorporating global branch history provides a substantial improvement over the simpler Bimodal predictor, while Perceptron gives a smaller additional improvement.

Q2. Does higher branch prediction accuracy always result in higher IPC?

Ans: No. As you can see in Gshare, the IPC is higher than that of Bimodal, but as accuracy increases from Bimodal to Perceptron, IPC has decreased. So branch prediction accuracy is important, but IPC is influenced by the entire processor execution, including cache/memory behavior and the cost of stalls. Higher branch prediction accuracy does not always result in higher IPC. For example, in 605.mcf_s, GShare achieves 91.82% accuracy compared with Bimodal's 90.05%, but its IPC of 0.3288 is slightly lower than Bimodal's 0.3298. Similarly, for 607.cactuBSSN_s, Perceptron has the highest accuracy,

while GShare has the highest IPC. This indicates that IPC depends not only on branch prediction accuracy but also on cache and memory behavior and other processor stalls.

Q3. How does branch misprediction affect processor performance?

Ans: Branch mispredictions negatively affect processor performance because a wrong prediction can cause incorrect speculative execution and subsequent recovery. This is reflected in the results for 602.gcc_s, where Bimodal has 19.82 MPKI and an IPC of only 0.2820, while Perceptron reduces MPKI to 8.68 and increases IPC to 0.3198. A similar relationship is observed for 605.mcf_s, where Perceptron reduces MPKI to 18.05 compared with 24.71 for Bimodal and achieves a higher IPC of 0.3483. However, the relationship is not perfectly proportional because other components of processor performance, particularly memory behavior, also contribute to IPC.

Q4. Do different predictors affect cache-related statistics?

Ans: The predictor changes substantially affect branch accuracy, but the cache statistics remain extremely close. The different branch predictors have only a small effect on cache-related statistics in these experiments. For example, the L1D miss rate for 602.gcc_s remains between approximately 8.03% and 8.27% across the three predictors, while the LLC miss rate remains approximately 99.75%. Similarly, 605.mcf_s shows L1D miss rates around 14.2–14.4% and LLC miss rates around 43%. Therefore, the major differences between the predictors are observed in branch prediction accuracy and IPC rather than cache miss behavior. Small cache differences can occur because changes in branch prediction alter the timing and flow of execution, but the cache configuration itself was unchanged.

Q5. Which characteristics of the workload influence predictor performance?

Ans: Workload characteristics such as branch frequency, the proportion of conditional branches, and the predictability of branch behavior influence predictor performance. 602.gcc_s and 605.mcf_s contain approximately 25.6% and 24.8% branch instructions, respectively, and therefore branch prediction has a significant effect on their performance. In contrast, 607.cactuBSSN_s contains only about 0.91% branch instructions and achieves over 99.3% accuracy with all three predictors. Thus, workloads with frequent and less predictable branches benefit more from sophisticated branch prediction mechanisms, while workloads with fewer or highly predictable branches show smaller differences between predictors.

Step-03: Implement a 2-bit branch predictor

```
Generating dependencies for replacement/llc_replacement.cc...
Compiling replacement/llc_replacement.cc...
Generating dependencies for prefetcher/kpcp_util.cc...
Compiling prefetcher/kpcp_util.cc...
Generating dependencies for prefetcher/l1d_prefetcher.cc...
Compiling prefetcher/l1d_prefetcher.cc...
Generating dependencies for prefetcher/l1i_prefetcher.cc...
Compiling prefetcher/l1i_prefetcher.cc...
Generating dependencies for prefetcher/l2c_prefetcher.cc...
Compiling prefetcher/l2c_prefetcher.cc...
Generating dependencies for prefetcher/llc_prefetcher.cc...
Compiling prefetcher/llc_prefetcher.cc...
Linking bin/champsim...

ChampSim is successfully built
Branch Predictor: two_bit
L1I Prefetcher: no
L1D Prefetcher: no
L2C Prefetcher: no
LLC Prefetcher: no
LLC Replacement: lru
Cores: 1
Binary: bin/two_bit-no-no-no-no-lru-1core
```

(two-bit BP)

Note: Bimodal is a natural comparison because it is also a 2-bit counter-based predictor. Also, we will be comparing our 2-bit with Bimodal.

3.1 EXPERIMENTAL TABLE:

Trace	Predictor	IPC	Accuracy	MPKI
602.gcc_s	Existing Bimodal	0.282009	92.2669%	19.8244
602.gcc_s	2-bit	0.282009	92.2669%	19.8244
605.mcf_s	Existing Bimodal	0.329782	90.0509%	24.7064

605.mcf_s	2-bit	0.328631	89.7756%	25.3899
607.cactuBSSN_s	Existing Bimodal	1.224590	99.3398%	0.0601
607.cactuBSSN_s	2-bit	1.224740	99.4200%	0.0528

3.2 ANALYSIS:

Q1. How is the predictor indexed?

Ans: uses a 1K-entry table of 2-bit saturating counters, indexed using: $\text{index} = \text{PC} \% 1024$
And, each counter has 4 states:

State	Meaning	Prediction
00	Strongly not taken	NT
01	Weakly Not taken	NT
10	Weakly taken	T
11	Strongly Taken	T

The 2-bit predictor uses the branch instruction's program counter (PC) to index a table containing 1024 2-bit saturating counters. The index is calculated as $\text{PC} \% 1024$. Each table entry therefore stores the prediction state associated with a group of branches mapping to that entry. Since the table has a finite number of entries, different branch PCs can map to the same counter, resulting in aliasing.

Q2. Why are two bits used instead of one?

Ans: A 1-bit predictor changes its prediction immediately after a branch outcome changes. It has learned T will predict the first N incorrectly, then switch to predicting N. When the branch goes back to T, it makes another mistake. But a 2-bit predictor is more resistant to such a single unusual outcome because it needs two consecutive contrary outcomes to move from a strong state to the opposite prediction. Thus, two bits are used to provide hysteresis in the prediction. Unlike a 1-bit predictor, a 2-bit predictor does not

immediately reverse its prediction after a single unexpected branch outcome. Two consecutive outcomes in the opposite direction are required to move from a strongly predicted state to the opposite prediction. This makes the predictor more stable for branches that are usually Taken or usually Not-Taken but occasionally behave differently.

Q3. How do the four states work?

Ans: The predictor contains four states: Strongly Not Taken (00), Weakly Not Taken (01), Weakly Taken (10), and Strongly Taken (11). States 00 and 01 predict Not Taken, while states 10 and 11 predict Taken. A Taken branch increments the counter toward 11, while a Not-Taken branch decrements it toward 00. The counter saturates at the two extreme states and therefore cannot overflow or underflow.

Q4. How does the predictor handle repeated Taken/Not-Taken patterns?

Ans: For repeated Taken or Not-Taken branches, the predictor quickly moves toward the corresponding strongly predicted state. For example, repeated Taken outcomes move the counter toward 11, while repeated Not-Taken outcomes move it toward 00. However, alternating or irregular patterns are more difficult because consecutive outcomes continually push the counter in opposite directions

Q5. Cases where the predictor performs poorly.

Ans: The predictor performs poorly for workloads containing irregular or difficult-to-predict branch behavior. This is evident for 605.mcf_s, where the predictor achieves only 89.7756% accuracy and 25.3899 MPKI. The limited 1K-entry table can also cause aliasing, where multiple branch PCs map to the same counter using $PC \% 1024$. Outcomes from different branches can therefore interfere with each other's prediction states. Alternating or irregular branch patterns are also difficult for a simple 2-bit counter to predict.

Step-04: Table-based Branch Predictor

To study how changing the size of your 2-bit counter table affects branch-prediction performance.

Table Size	Trace	IPC	Branch Accuracy (%)	MPKI	Misprediction Rate (%)	
2K	602.gcc_s	0.282009	92.2669	19.8244	7.7331	
2K	605.mcf_s	0.329169	89.9752	24.8943	10.0248	
2K	607.cactuBSSN_s	1.22474	99.4189	0.0529	0.5811	

Table Size	Trace	IPC	Branch Accuracy	MPKI	Misprediction Rate
4K	602.gcc_s	0.282009	92.2669%	19.8244	7.7331%
4K	605.mcf_s	0.329188	89.9781%	24.8871	10.0219%
4K	607.cactuBSN_s	1.224730	99.4178%	0.0530	0.5822%

Table Size	Trace	IPC	Branch Accuracy (%)	MPKI	Misprediction Rate (%)	
------------	-------	-----	---------------------	------	------------------------	--

8K	602.gcc_s	0.2820 09	92.2669	19.82 44	7.7331	
8K	605.mcf_s	0.3292 49	89.987	24.86 49	10.013	
8K	607.cactuBS SN_s	1.2247 3	99.4189	0.052 9	0.5811	

Table Size	Trace	IPC	Branch Accuracy (%)	MPKI	Misprediction Rate (%)	
16K	602.gcc_s	0.2820 09	92.2669	19.82 44	7.7331	
16K	605.mcf_s	0.3292 49	89.987	24.86 49	10.013	
16K	607.cactuBS SN_s	1.2247 3	99.4189	0.052 9	0.5811	

(all .csv files attached in repo + one common results file)

Increasing the predictor table size does not produce the same effect for all workloads. For 602.gcc_s, the branch prediction accuracy remains unchanged at 92.2669% from 1K to 16K entries. For 605.mcf_s, increasing the table size improves accuracy from 89.7756% at 1K to 89.9870% at 16K, with most of the improvement occurring between 1K and 2K. For 607.cactuBSSN_s, the accuracy remains approximately 99.42% for all table sizes. Therefore, the benefit of increasing predictor capacity depends on the workload's branch behavior and the amount of aliasing present in the predictor table.

The effect of table size on IPC is relatively small for the tested workloads. The largest change is observed for 605.mcf_s, where IPC increases from 0.328631 at 1K entries to 0.329249 at 16K entries. In contrast, 602.gcc_s remains at an IPC of 0.282009 across all table sizes, while 607.cactuBSSN_s remains approximately 1.2247. This indicates that

increasing predictor capacity does not necessarily translate into a significant performance improvement once the existing table is sufficient for the workload.

The improvement eventually saturates because increasing the number of entries reduces aliasing only when multiple active branches are competing for the same predictor entries. Once the table is sufficiently large for the workload, further increases provide little additional benefit. This behavior is particularly visible for 605.mcf_s, where accuracy improves noticeably from 1K to 2K but changes only slightly from 2K through 16K.

The table-based 2-bit branch predictor was evaluated using 1K, 2K, 4K, 8K, and 16K entries while keeping the simulation configuration unchanged. The results show that the effect of table size is workload dependent. For 602.gcc_s, branch accuracy and IPC remained unchanged across all table sizes. For 605.mcf_s, increasing the table size improved accuracy from 89.7756% at 1K to 89.9870% at 16K and increased IPC from 0.328631 to 0.329249. However, most of this improvement occurred between 1K and 2K, after which the gains became very small. For 607.cactuBSSN_s, accuracy remained approximately 99.42% and IPC remained approximately 1.225 for all table sizes. These results demonstrate that increasing predictor table capacity can reduce aliasing, but its benefit depends on the workload. Once the table is sufficiently large, additional entries provide little or no measurable improvement.

SOURCE CODE:

<https://github.com/tanishka-trivedi/Computer-Architecture-LAB03>